Contents

Chapter 1

Unit legacey.random

1.1 Description

A very simple random number generator for Blaise pascal. Not Needed for Delphi or FPC compilers

1.2 Overview

`random`

1.3 Functions and Procedures

random

Declaration `function random: Double;`

Description Returns a random double from 0 to 1.0

1.4 Variables

RandSeed64

Declaration `RandSeed64: UInt64 = 88172645463325252;`

Description Random number sequence will always be the same with same seed! If it is a problem, a seed can be used from system clock.

Chapter 2

Unit morse.base

2.1 Description

The base unit for the Morse-lib. These are what are needed to configure the morse code generator. A default settings is provided called `DefaultMorseSettings`(**??**). The settings are as follows :

```
  WPM : 18
 Farnsworth : Disabled
 Noise : Disabled
 Tone Hz  : 800 Hz
 Volume : (TBA)
```

2.2 Overview

`TmorseSettings Record`

`TElementTimings Record`

2.3 Classes, Interfaces, Objects and Records

TmorseSettings Record

Description

The settting used for creation of the morse code sounds

Fields

wpm `public wpm:  Integer;`

Set the WPM for the morse code output

ToneHertz `public ToneHertz: Integer;`

Set the tone in hertz for the audio output

morseLevel `public morseLevel: Integer;`

Set the level of the output. A good start would be 70% of max Integer value

FarsworthEnabled `public FarsworthEnabled: Boolean;`

Enable Farnsworth Spacing

FarnsworthWPM `public FarnsworthWPM: Integer;`

Set Farnsworth Spacing WPM, if enabled, otherwise, no affect

NoiseType `public NoiseType: TnoiseType;`

Set the noise type to off, white, or pink

NoiseLevel `public NoiseLevel: Integer;`

Set the noise level. It should be the inverse of the morseLevel, 30 % of max Integer value, for example

TElementTimings Record

Description

Holds all the timing values for the elements

Fields

dit `public dit: Single;`

The length of a dit in fractional seconds

dah `public dah: Single;`

The length of a dah in fractional seconds is dit * 3

intraElement `public intraElement: Single;`

The length of time between elements

interCharacter `public interCharacter: Single;`

The length of time between characters. It can change based on Farnsworth settings. Otherwise it is dit * 3

interWord `public interWord: Single;`

The length of time between words. It can change based on Farnsworth settings. Otherwise it is dit * 6

2.4 Types

Tpcm

Declaration `Tpcm = array of Integer;`

Description Array used for all PCM Data in morse-lib

EmorseLibError

Declaration `EmorseLibError = Exception;`

Description Used for all exceptions in morse-lib

TsampleRate

Declaration `TsampleRate = (...);`

Description sample rates for PCM data

Values `srLow = 8000`
`srMedium = 22050`
`srCD = 44100`
`srDVD = 480000`

TnoiseType

Declaration `TnoiseType = (...);`

Description Noise types for audio PCM data (off, white, or pink)

Values `ntOff`
`ntWhite`
`ntPink`

2.5 Variables

DefaultMorseSettings

Declaration `DefaultMorseSettings: TmorseSettings;`

Description A preset default of morse sounds at 18WPM, no noise, no Farnsworth Spacing

Chapter 3

Unit morse.dictionary.ascii

3.1 Description

Contains the asciiToMorse array and setup for it from the MorseDictionary

3.2 Variables

asciiToMorse

Declaration `asciiToMorse: array [0..127] of string;`

Description use asciiToMorse to quickly get the morse code string of an ascii character. Example `asciiToMorse[Ord(a)]` shoud output a string of '.-', the dit-dah representing the letter A in morse code

Chapter 4

Unit morse.dictionary.util

4.1 Description

A class Tdictionary<String, String> that returns the Value of the dit '.' or dah '-' of the given ASCII character Capital letters only!

```
MorseLookup['A']
```

Should return a string type with '.-'
The class is initialized and free when the unit is used in a program or library.

4.2 Variables

MorseLookup

Declaration `MorseLookup: TDictionary<string, string>;`

Chapter 5

Unit morse.parser.Util

5.1 Overview

`TmorseParser Class`

5.2 Classes, Interfaces, Objects and Records

TmorseParser Class

Hierarchy

TmorseParser > TObject

Methods

inputString

Declaration `public procedure inputString(const ATextInput:  string);`

getOutput

Declaration `public function getOutput:  string;`

Chapter 6

Unit morse.pcm.pinknoise

6.1 Overview

`PinkNoise`

6.2 Functions and Procedures

PinkNoise

Declaration `procedure PinkNoise(var aPCM: TNoise; aLengthms: UInt64; aAmplitude: UInt16 = 27000; aSampleRate: UInt32 = 44100);`

Description Generates pink noise (1/f noise) with specified parameters

Parameters **aPCM** - Output array that will contain the generated samples

aLengthms - Duration of the noise in milliseconds

aAmplitude - Peak amplitude (default: 27000, max safe: 32767)

aSampleRate - Sample rate in Hz (default: 44100)

6.3 Types

TNoise

Declaration `TNoise = array of Integer;`

Chapter 7

Unit morse.pcm.sine

7.1 Overview

`sineWave`

`msToSamples`

7.2 Functions and Procedures

sineWave

Declaration `procedure sineWave(var APcm : Tpcm; const length_ms : Integer; const Hertz : Integer = 800; const Amplitude : Integer =1653562408; const sampleRate: Integer = 8000);`

Description type Tpcm = array of Integer;

msToSamples

Declaration `function msToSamples(const ms: Integer; const sampleRate: Integer) : Integer; inline;`

Chapter 8

Unit morse.pcm.taper

8.1 Description

This unit contains the code to taper the audio ends of the morse code pulse. Ends are tapered to 5 milliseconds

8.2 Overview

`taper`

8.3 Functions and Procedures

taper

Declaration `procedure taper(var Apcm:  Tpcm; sampleRate :  Integer = 8000);`

Description Pass the Tpcm and the sample rate. Default sample rate is 8000. Tpcm is modified. Do not send Tpcm with silence at end . Audio must start and stop at ends of Tpcm to be tapered properly.

Chapter 9

Unit morse.pcm.Util

9.1 Overview

`TElementSettings Record`

`TElementProperties Class`

`TCreateElement Class`

`TElementPCM Class`

9.2 Classes, Interfaces, Objects and Records

TElementSettings Record

Description

TNoiseSettings = record noiseType :TNoiseType; noiseLevel : single; //0.1 to 1 end;

Fields

SampleRate `public SampleRate: UInt32;`

FreqHz `public FreqHz: UInt32;`

MorseTimings `public MorseTimings: ImorseTiming;`

noiseType `public noiseType:TNoiseType;`
Noise : TnoiseSettings;

noiseLevel `public noiseLevel: single;`

TElementProperties Class

Hierarchy

TElementProperties > TObject

Description

maybe a class that calculates on creation?

Properties

sampleRate `public property sampleRate : integer read fSampleRate write fSampleRate;`
fNoise : TnoiseSettings; fGenerator : TElementPCM

TCreateElement Class

Hierarchy

TCreateElement > TObject

Methods

create

Declaration `public constructor create(settings : TElementSettings);`

TElementPCM Class

Hierarchy

TElementPCM > TObject

Description

NOTE ========== Submit Bug report for Blaise. It does not catch mismatch method declarations in class!
==========

9.3 Types

EMorseElement

Declaration `EMorseElement = Exception;`

Description , morse.pcm.pinknoise, legacey.random

TNoiseType

Declaration `TNoiseType = (...);`

Description TpcmArray<T> = array of T; //Not Accepted in Blaise yet!

Values `nsOff`

`nsWhite`

`nsPink`

`nsGaussien`

Chapter 10

Unit morse.pcm.whitenoise

10.1 Overview

`whiteNoise`

10.2 Functions and Procedures

whiteNoise

Declaration `procedure whiteNoise(var aPCM: Tpcm; aLengthms:  uint64; aAmplitude: Integer = 1653562408; aSampleRate:  uint32 = 44100);`

Description type TNoise = array of int16;

Chapter 11

Program test